Computational companion to Cartier–Miller evaluation

David Jordan
David.jordan@mu.ie

Revision 22 October 2026

Abstract

This report accompanies the 22 October 2026 Cartier–Miller manuscript. It collects the single-prime rounded-third boundary methods, the joint recurrence for tables across primes, and the complete historical benchmark descriptions. The historical implementations and timing campaigns are identified separately from the later uniform curve-order evaluator. No timing is transferred to the newer evaluator. The main paper contains the coefficient, weighted-sum and de Rham proofs; this report records the computational methods and evidence.

1 Outputs and implementation versions

Equation references prefixed by M refer to the numbered equations of the accompanying main paper; unprefixed equation numbers belong to this report.

All coefficient and character outputs are residues modulo the supplied prime. A complete binomial query includes both its strict prefix and its boundary. A single-pole character query returns $C_f(a) = \sum_{x \neq a} \chi(f(x))/(x-a)$. These outputs must be matched before comparing runtimes. Trace acquisition is charged where the experiment declares a cold query; prepared and cached workloads have different contracts.

The historical quarter-point pilot, rounded-third integration and character-sum campaign used different code revisions, hosts and protocols. Their raw measurements are retained separately. The preferred evaluator introduced after those campaigns uses one base-field chain at the normalized twist order; it has correctness evidence but no timing campaign in this report. The supplied new weighted adapter likewise has correctness checks, rather than inherited benchmark timings.

For $p \geq 11$, $p \equiv 2 \pmod 3$, put $m = (p-2)/3$, $L = m+1$, $a_j = \binom{2j}{j} 8^{-j}$, $B_j = \sum_{i<j} a_i$, and $G = m! \bmod p$. The main paper proves

$$U_p(j) = 4B_j - 2j(2j+5)a_j, \qquad a_L = \frac{3}{2G^3}, \qquad G = \left(\frac{3}{2a_L}\right)^{(2p-1)/3}. \tag{1}$$

It also proves $U_p(L) = 4B_L - (17/3)G^{-3}$, $U_p(m) = 4B_L - (16/3)G^{-3}$, and $a_m = -4a_L$. Its zero-trace elliptic query supplies B_L in $O(\log p)$ field operations. The remaining single-prime boundary is acquired by the methods below. A dense prime table can acquire prefix and boundary jointly through a different recurrence.

2 Boundary acquisition at one prime

The integrated single-prime routine validates p, obtains B_L from the unchanged differentiated Miller provider, acquires the signed boundary a_L by the selected backend, and assembles (M.49). Formula (M.52) recovers G when the backend supplies a_L directly; (M.54) then supplies the adjacent outputs. Apart from boundary acquisition, the complete routine requires $O(\log p)$ field operations, including

modular powering. It returns the same tuple $(B_L, a_L, U_p(L), G)$ for every backend. The implemented public interface checks primality exactly in the supported domain $11 \le p < 2^{63}$, $p \equiv 5 \pmod 6$.

2.1 Certified Jacobi boundary acquisition

Write $k_\ell = (p-1)/\ell$ for a prime conductor $\ell \mid p-1$. The bounded backend, called Jacobi reconstruction here (`T3` in the code), supports primes $5 \le \ell \le 89$, with $k_\ell \ge 4$ required for forced calls. It reconstructs a Jacobi sum in $\mathbb{Z}[\zeta_\ell]$ by lattice reduction, accepts it only after the algebraic and normalization certificates, and evaluates its Galois conjugates to obtain grid-factorial relations. In particular, the norm relation $J\overline{J} = p$ is combined with the required root-of-unity normalization; a norm check alone is not the full certificate. Integer relation plans are checked on use. Two interval products connect the grid to G and to a rounded-third anchor. The interval lengths are approximately $k_\ell/3$ and $k_\ell/6$, or $k_\ell/3$ and $k_\ell/12$ when the denominator-12 anchor is available for $p \equiv 5 \pmod{12}$. The companion gives the exact residue-class formulas, relation plans, normalizations and certificates (Jordan, 2026a).

The denominator-12 anchor means an auxiliary factorial at $n_{12} = \lfloor (p-5)/12 \rfloor$ or $5n_{12} + 2$, corresponding to rational fractions with denominator 12. It does not assert $12 \mid p-1$; only the prime Jacobi conductor ℓ must divide $p-1$. The saving lies in the anchor interval: the rounded-third relations replace the longer grid normalization by a shorter anchor tied to the same target. The signed half-factorial is not required, and neither class numbers nor GRH enter this integrated route. These are bounded specializations of Jacobi and modular-factorial methods, not a new general complexity theorem. For fixed ℓ, the interval work retains square-root dependence on p up to logarithmic and multiplication factors. Lattice reconstruction is not proved to succeed on every input; exact acceptance and guaranteed reconstruction are different claims.

The automatic policy uses native FLINT factorial below 10^7. Above that threshold it ranks eligible conductors with $k_\ell \ge 10^4$ by a recorded cost model and attempts Jacobi reconstruction when its prediction beats the nodal alternative. Certified reconstruction misses are recorded; automatic mode tries the remaining candidates and then falls back to the exact nodal backend. Forced Jacobi reconstruction raises an error if reconstruction cannot be certified. The threshold is an implementation policy, not a universal crossover. The nodal backend is the companion's factorial-ratio anchor evaluated by FLINT block products; it uses no class number. Native factorial remains an explicit comparator and alternative. This fallback policy preserves the mathematical output but does not guarantee the fastest route on every machine.

2.2 Remainder forests for dense prime tables

A remainder forest shares ordered integer products and their reductions across many primes. Product trees combine recurrence intervals and prime moduli; remainders descend to the leaves to recover the required product at each prime's cutoff. Processing successive smaller trees, with a carry reduced modulo the unprocessed primes, controls the size of the retained arithmetic state. This is established accumulating remainder-forest machinery (Harvey & Sutherland, 2016), specialized here to an upper-triangular recurrence.

Over the rationals put $\Delta_j = 4^j j!$, $\widehat{B}_j = \Delta_j B_j$ and $\widehat{a}_j = \Delta_j a_j$. Then

$$\mathsf{A}_j = \begin{pmatrix} 4(j+1) & 4(j+1) \\ 0 & 2j+1 \end{pmatrix}, \qquad \mathsf{A}_{j-1} \cdots \mathsf{A}_0 = \begin{pmatrix} \Delta_j & \widehat{B}_j \\ 0 & \widehat{a}_j \end{pmatrix}. \tag{2}$$

This follows from $B_{j+1} = B_j + a_j$ and the ratio preceding (M.48); the empty product is the identity. At $j = L < p$, division by the nonzero Δ_L recovers B_L and a_L, while $G = \Delta_L / \left(4^L L\right)$ and (M.49) recover the other outputs. Thus the forest computes the complete tuple jointly and bypasses the Miller pass. Its gain is batch amortization, not an acceleration of an isolated Miller query. The companion provides ordered-product and carry invariants, bit-size and storage bounds, and reproduction commands (Jordan, 2026b, 2026a).

Dense prime schedules share enough recurrence work to benefit strongly. A sparse schedule still pays for products from index zero to its last requested cutoff, so independent calls can be preferable. The experiments compare these workloads separately. The integrated single-prime adapter does not automatically replace the workbench's existing many-prime dispatcher.

2.3 Supported evaluation modes

Table 1. Supported evaluation modes.

The table distinguishes the geometric query from any additional boundary arithmetic.

Evaluation	Prefix or coefficient sum	Additional arithmetic
General admissible genus-one sum	Differentiated Miller query	Exact trace preparation
Original quarter-point weighted sum	Cartier–Miller prefix	Classical CM boundary and trace data
Rounded-third sum at one prime	Cartier–Miller prefix with known zero trace	Native factorial, nodal or certified Jacobi boundary
Rounded-third table across primes	Joint recurrence through a remainder forest	Shared prefix and boundary acquisition

For suitable individual primes the integrated Jacobi route improves complete evaluation, while dense tables favour the retained forest. Tables 3–5 give the matched rounded-third measurements; Table 6 gives a separate character-sum comparison.

3 Historical computational benchmarks

3.1 Original quarter-point pilot

Table 2 presents a quarter-point pilot for seven primes $p \equiv 1 \pmod 4$, with $n = (p-1)/4$. Both implementations compute B_n, a_n and $U_p(n)$, including per-input preparation. The comparator is a standalone C++ adapter to the unchanged SageMath 10.8 Harvey `hypellfrob` interval-product kernel, which uses BGS-based recurrence techniques (Bostan et al., 2007; Harvey, 2007; SageMath Developers, n.d.). This is a benchmark of that recurrence task, not of Harvey's full Kedlaya algorithm.

Table 2. Complete quarter-point evaluation in milliseconds: Python Cartier–Miller versus the C++ Harvey/BGS recurrence adapter, including per-input preparation

Prime p	Stopping index n	Cartier–Miller Python	Harvey BGS C++
97	24	0.05	0.01
1,009	252	0.06	0.05
10,009	2,502	0.07	0.10
100,049	25,012	0.09	0.54
1,000,033	250,008	0.10	1.78
10,000,121	2,500,030	0.11	4.44
100,000,037	25,000,009	0.29	16.19

The run used a shared hosted Linux environment reporting an AMD EPYC 9V74 processor, Python 3.12.14, GNU C++ 13.3.0 with `-O3`, and NTL 11.5.1 configured for one thread. Times are medians of 11 per-call timing-batch means, with alternating method order in warmed persistent processes. Batches targeted about 25 ms and were capped at 2,000 evaluations. Answers and per-call mathematical data were recomputed; library tables could remain warm. Validation, compilation, primality testing, process startup, communication and output writing were excluded. Host load and effective CPU frequency were not controlled.

All timed outputs agreed. Adapter validation additionally covered 10,809 stopping-point queries across 92 primes from 7 to 499, and 328 quarter-point comparisons for primes $p \equiv 1 \pmod 4$ from 13 to 4,999, with no mismatches. The timing run did not change these validation counts.

Cornacchia/Gauss preparation includes a nonresidue scan; this pilot does not implement Schoof setup or establish Corollary 8 of the main paper's worst-case bound. The direct characteristic-p proof did not change the evaluator. Small-input entries include call and loop overhead. Seven selected inputs, different implementation languages, an uncontrolled host and unmeasured memory limit general speed or scaling claims. Raw samples and reproduction commands accompany the manuscript.

3.2 Integrated rounded-third single-prime benchmarks

Tables 3–5 compare methods returning the identical complete tuple $(p, B_L, a_L, U_p(L), G)$. Each cell gives the median and observed minimum–maximum range of three samples with rotated method order on one host, using Python 3.12.14, python-flint 0.9.0 (FLINT 3.6.0), gmpy2 2.3.2 (GMP 6.3.0), Linux x86-64 and an AMD EPYC 9V74 processor. No concurrent number-theory job ran during timing; CPU frequency and scheduling were not controlled. These rounded-third measurements use a different protocol from the quarter-point pilot and must not be pooled with Table 2.

Each single-prime call rebuilds its Jacobi context and product trees and reruns Miller arithmetic; complete answers are not cached. Primality checks, registry and provider setup, conductor discovery, reconstruction certification, interval products, normalization, derived G and result collection are charged. Imports and prime selection are excluded, and reference comparisons occur after timing. A first call can load a prime-independent plan table or native-library handle. The native comparator calls FLINT's `n_factorial_fast_mod2_preinv` (FLINT developers, n.d.) for $m!$ and uses the same Miller prefix. The nodal comparator uses the same prefix with its factorial-ratio boundary.

Table 3. Complete rounded-third evaluation in milliseconds, shown as median [minimum, maximum] of three calls. Auto is the integrated automatic route; ℓ is its selected conductor.

Prime p	Auto route ℓ	Native + Miller	Nodal + Miller	Auto + Miller	Native / auto
70,000,283	Jacobi 11	11.244 [11.013, 14.510]	9.716 [9.197, 11.064]	6.745 [6.105, 8.968]	1.67
70,000,481	Jacobi 11	11.351 [11.004, 11.462]	9.547 [9.542, 11.833]	5.879 [5.386, 5.905]	1.93
70,000,613	Jacobi 11	15.190 [14.775, 15.740]	12.261 [10.969, 12.432]	7.868 [6.523, 8.340]	1.93
7,000,000,013	Jacobi 31	261.129 [254.152, 264.945]	220.053 [215.825, 251.297]	58.941 [56.682, 60.779]	4.43
7,000,000,277	nodal	256.361 [254.963, 259.401]	214.941 [208.206, 217.394]	214.746 [212.828, 215.143]	1.19
7,000,000,727	nodal	260.154 [255.525, 268.183]	214.452 [209.942, 215.550]	214.251 [212.405, 217.044]	1.21
7,000,000,769	nodal	263.522 [258.686, 265.291]	217.327 [212.077, 225.342]	225.696 [213.945, 226.674]	1.17
7,000,002,851	Jacobi 23	259.000 [258.844, 273.559]	214.838 [213.861, 219.622]	63.885 [62.825, 65.143]	4.05
7,000,011,929	Jacobi 13	266.407 [263.097, 283.127]	232.299 [218.851, 235.606]	77.799 [76.285, 85.781]	3.42
700,000,000,031	Jacobi 71	5894.902 [5816.792, 6078.298]	4909.134 [4835.276, 4939.634]	1153.621 [1147.457, 1310.309]	5.11
700,000,001,633	Jacobi 47	5833.161 [5831.019, 5857.249]	4760.048 [4749.239, 4880.084]	895.216 [868.507, 900.682]	6.52

Prime p	Auto route ℓ	Native + Miller	Nodal + Miller	Auto + Miller	Native / auto
700,000,004,549	Jacobi 31	5751.378 [5718.851, 6103.010]	4784.071 [4749.360, 4909.331]	1014.534 [996.811, 1093.955]	5.67

All complete outputs agreed. At the nine inputs selecting Jacobi, speedups over native factorial plus Miller ranged from 1.67 to 6.52. At $p = 700{,}000{,}001{,}633$, the complete median fell from 5.833161 s to 0.895216 s; the Miller prefix contributed approximately 0.48% of the new runtime. The no-small-factor inputs selected nodal fallback, so their gains over native are nodal gains, not Jacobi acceleration. Against direct nodal calls, automatic fallback has small selection overhead and timing variation.

3.3 Dense and sparse prime schedules

For Table 4 every method evaluates all admissible primes below the same exclusive bound. Sieve, arithmetic, collection and canonical tuple hashing are charged; disk I/O is excluded. Python and GMP forests use the same retained recurrence with different integer backends and acquire prefix and boundary jointly. Auto selects native throughout these small-prime ranges, so no dense-table gain is attributed to Jacobi.

Table 4. Dense rounded-third tables, complete seconds as median [minimum, maximum] of three runs.

Exclusive bound	Tuples	Native + Miller	Auto + Miller	Python forest	GMP forest
2,000	152	0.120307 [0.120265, 0.122512]	0.122290 [0.120004, 0.126363]	0.001377 [0.001267, 0.001557]	0.001255 [0.001231, 0.001441]
20,000	1,135	1.273731 [1.272739, 1.344891]	1.336106 [1.278975, 1.344524]	0.015224 [0.013977, 0.016268]	0.011338 [0.010396, 0.017316]
100,000	4,805	6.677996 [6.650758, 6.870394]	6.652261 [6.646842, 6.750717]	0.157454 [0.155444, 0.161275]	0.057797 [0.056992, 0.057938]

At the 100,000 bound the GMP forest is approximately 115.5 times faster than independent native-factorial/Miller calls. This is the retained batch algorithm's amortization, not a speedup of the Miller pass. The forest need not be preferable for sparse schedules: it pays to construct recurrence products from index zero, rather than receiving the initial state near the first requested prime for free.

Table 5. One matched sparse schedule of sixteen primes from 1,000,211 to 2,501,171, complete runtime in seconds.

Method	Runtime in seconds
Native factorial + Miller	0.037191 [0.036596, 0.038203]
Automatic integration	0.037016 [0.036621, 0.039312]
Forced Jacobi $\ell = 5$	0.039155 [0.038894, 0.041857]
Python forest	8.171517 [7.951387, 8.624335]
GMP forest	0.689112 [0.679250, 0.817014]

The exact same sixteen-prime schedule and complete outputs were used throughout, with all output hashes agreeing. Independent native calls took 0.037191 s, versus 0.689112 s for the GMP forest,

making the forest approximately 18.5 times slower here. Forced Jacobi at $\ell = 5$ did not improve this moderate-prime workload. Very large sparse-prime forest requests were outside the declared recurrence-cutoff cap of 2,000,000 and received no hypothetical timing.

The practical conclusion is workload-dependent: retain independent factorial and nodal alternatives, use certified Jacobi acquisition on suitable larger individual primes, and exploit a forest for dense tables. These experiments do not compare an optimized compiled forest, unmodified Tal reconstruction, the earlier project Jacobi backend, or complete PARI, SageMath and Magma systems. They establish improvements over the named matched comparators, not superiority over every available tool. The observed ranges are descriptive, not confidence intervals; in particular, modest fallback differences should not be interpreted as a robust universal speedup.

The reproducibility archive contains integrated.py, the original-definition oracles, the frozen selection rules, all 17 confirmation inputs, raw benchmark samples, the million-bound survey and preserved supporting proofs. README.md gives the commands for confirmation, benchmarks, survey and hash audit. The final file manifest and the earlier internal freeze have distinct roles: the freeze records the candidate before confirmation, whereas the manifest checks the delivered package. The standalone adapter integrates the existing boundary registry and complete-sum function; it does not constitute an updated graphical workbench release or a new many-prime dispatcher (Jordan, 2026a).

3.4 Matched character-sum workloads

This is a separate experiment with the identical residue $C_f(a)$, or identical multi-pole weighted sum, as output for every method in a comparison (Jordan, 2026c). Table 6 records medians for a cold mathematical instance: a fresh prime, curve and pole for each repetition, with trace preparation charged to the historical Python Cartier–Miller implementation and no trace charged to the recurrence comparators. Imports, interpreter start, prime selection and PARI initialization were excluded. Outputs were checked outside the timed region.

Table 6. Cold single-pole character-sum medians in milliseconds.

Within each cell, cubic / quartic. A dash means the method was not run at that class, not an extrapolated runtime. The native column is the best measured linear method in that class (C recurrence, C enumeration or FLINT polynomial power).

Bits	Historical Cartier–Miller including trace	Harvey/BGS adapter	Best native linear
10	0.138 / 0.169	0.561 / 0.610	0.033 / 0.040
20	0.246 / 0.239	6.34 / 14.78	25.04 / 38.93
30	0.367 / 0.356	170.21 / 474.16	—
40	1.18 / 0.983	6,900 / 16,000	—
62	131.73 / 133.69	—	—

The uncontrolled Linux host reported an Intel Xeon at 2.10 GHz, two virtual CPUs and 7.8 GiB RAM. Software was CPython 3.13.16, GCC 13.3.0 with `-O2`, python-flint 0.9.0/FLINT 3.6.0, cypari2 2.2.4/PARI 2.17.2 without SEA data, and the Harvey `hypellfrob` 2.1.1 kernel through passagemath-ntl 10.8.13. Workers were single-threaded and methods ran sequentially in seeded random order. Pilots chose repetition counts targeting roughly 40 ms per block, capped at 200 repetitions and nine blocks; slower calls used fewer repetitions. The archive retains every block, repetition count, input class, validation status and budget decision.

These results support a sparse-query advantage for the named implementation on the tested inputs. They do not compare every possible implementation or prove universal crossover values. The square-root comparators had a declared 40-bit ceiling and were not run above it. PARI trace preparation used its available CM or baby-step/giant-step paths, so these are not SEA or Schoof timings. The separate trace-only class medians were approximately 122/117 ms at 62 bits; they came from other fresh instances and are not an exact decomposition of the cold medians. The uniform-order evaluator

of Theorem 4 of the main paper was developed after these measurements and has no new timing claim here. In the known-trace exceptional family, a direct Voloch–Semaev adapter was also faster than the historical candidate by approximately 2.0–5.7 times, reflecting implementation and adapter overhead with the same asymptotic short-chain cost.

Reuse changes the comparison. At 16 bits a prepared full-polynomial route overtook repeated historical scalar queries between 8 and 64 poles for cubics, and between 64 and 512 for quartics. At 20–23 bits the historical evaluator was fastest at every tested query count up to 512. Dense-query tests on 12–18-bit curves instead favoured all-poles correlation or polynomial multipoint evaluation, by approximately 9–147 times over the tested Python scalar loops. For a fixed excluded set with only residue coefficients changing, all methods used the same cached dot product; at eight poles the measured coefficient stage was approximately 0.7–1.1 microseconds. Thus a speed claim must specify the output, setup and reuse regime.

For one weighted sum at 20 bits and up to 256 poles, the historical evaluator was the fastest measured route; the tested polynomial method also paid 55–135 ms of preparation. Conversely, the tested Python second-kind branch implementation took roughly one second at 20 bits, where the trace-free BGS branch adapter took approximately 5–12 ms. No logarithmic branch-query claim follows from the nonbranch theorem.

The benchmark archive documents a host restart, reruns of only the missing jobs with the unchanged harness, and deterministic reconstruction of missing case metadata. Thirteen polynomial or multipoint jobs at 23 bits exceeded a declared 512 MiB working-memory target; that target was not enforced at runtime. They were flagged and none was the fastest in its cell. They supply no evidence of performance within that target. The source freeze covered the specified candidate files, not the entire timing and postprocessing harness. Independent review recomputed the displayed cold-query medians and ran bounded correctness comparisons; it did not rerun the full timing campaign. Raw data and these limitations remain part of the reproducibility record. These character-sum measurements must not be pooled with the earlier quarter-point or rounded-third protocols.

4 Rounded third validation records

The earlier rounded-third companion has a separate validation record (Jordan, 2026b). Its anchor suite checked all 152 admissible primes below 2,000 and 595 reconstructed conductor values against direct character enumeration. Its native-FLINT comparison comprised 30 complete evaluations over five selected primes. Its million-bound standard-library batch had independent prefix checks, complete checks below 20,000 and 40 reproducible larger complete checks. These inherited records are distinct from the integration checks below; the unshared table below 100 million is not included in the independently verified evidence.

The integrated implementation was internally recorded with a source digest before selecting fresh confirmation inputs; the commitment has no external timestamp. Development checked all 1,135 admissible primes below 20,000, with 3,405 complete method comparisons, 1,135 independent integer-binomial checks, 1,135 independently coded elliptic-prefix comparisons and 200 forced T3 checks. Three forest chunk counts agreed. Invalid domains and conductors, $p = 17$ and injected reconstruction misses were checked. The two relevant inherited test modules also passed, comprising 19 test methods.

Confirmation passed at 17 distinct primes, excluded from the recorded prior development, confirmation and benchmark ledgers, reaching $p = 7{,}000{,}000{,}009{,}553$. Every case compared complete native-factorial, nodal and automatic outputs and compared the Miller prefix with an independently coded elliptic provider. Applicable cases also used forced T3. The three approximately five-million inputs additionally received direct original weighted-recurrence, independent scaled-binomial-prefix and modular-binomial-product checks. At larger primes the original linear-time weighted recurrence was not run; agreement there uses independent prefix code and native-factorial boundary checks.

An exploratory forest survey produced all 39,264 admissible tuples below one million. The nor-

malization and adjacent-sum residuals held throughout; neither $U_p(L)$ nor $U_p(m)$ vanished in that range. All 1,135 rows below 20,000 and 64 seeded rows above that bound received independent definition-level checks; the remaining rows received internal identity checks. The nonexistence of simultaneous zeros follows separately from (M.54). This survey is newly executed evidence on a previously studied range, not disjoint confirmation. The integration package records the selection rules, hashes, exact outputs and raw measurements (Jordan, 2026a).

5 Reproduction and evidence

The original raw outputs, source commitments, case metadata and analysis scripts remain in the associated reproducibility archives. The 21 October character-sum supplement consolidates those records with the uniform-order implementation and weighted adapter. Its verification command checks historical source digests and recomputes the displayed cold-query medians from validated raw samples; this is not a new timing campaign.

Run the main revision's verification script to check the new truncation recovery and the stated de Rham coefficient relations. The release notes identify the authoritative uploaded manuscript, reconstructed source baseline, retained historical tables and the automatic numbering map. The report describes implementation evidence and does not claim guaranteed reconstruction for every Jacobi input or a uniform crossover between workloads.

AI assistance

Generative AI assisted the mathematical exploration, implementations, checking and preparation under the author's direction. The report and its computational records are unrefereed.

6 Verification of the revised coefficient adapter

The separately supplied `verification/cm_basefield.py` implements the uniform base-field chain, coefficient recovery and the pairwise excluded-domain adapter. It accepts a supplied exact projective trace. Its Hasse-bound and endpoint checks do not certify that trace. Its primality validator supports only $5 \le p < 2^{64}$; the mathematical theorem is not restricted to machine-word primes. Active branch poles raise an explicit unsupported-case exception. Zero-residue branch exclusions remain permitted. The faster polynomial-tree corrections are proved in the main paper but are not implemented in this reference adapter.

The independent `verify_revision.py` was rerun under CPython 3.12.14 and reproduced 6,489 truncation comparisons on 240 models over 15 primes from 5 to 59, together with the de Rham coefficient checks at 12 primes. The newer `check_adapter.py` compares the executable adapter with independently enumerated characters and actual pairs (x, y), and with polynomial powering. It uses different development samples from that first verifier.

Development used seed 2026102201 and the 15 primes from 5 to 59 listed in `ADAPTER_DEVELOPMENT.json`. There were 240 models, equally divided between cubics and quartics, 6,693 nonbranch single-pole queries, 6,693 truncation queries, and 6,960 all-pole polynomial identities. Its 1,440 weighted queries gave 4,320 output comparisons. There were zero mismatches.

The final candidate was then frozen with SHA-256

```
3d84b3fa3ef8b8383f31023a5a5d759f8
e1573fbcfc0d2b47c6e53da34eb00ed
```

before selecting seed 2026102202 and confirmation primes $61, 67, 73, 79, 83, 89, 97, 101, 103$. No candidate code changed after freezing. Confirmation used 144 models, equally divided between degrees, and passed 11,905 nonbranch single-pole queries, 11,905 truncation queries, 12,048 all-pole polynomial identities and 2,592 weighted output comparisons from 864 queries. All confirmation

primes are disjoint from the adapter development primes. The nine invalid-input controls were deliberately repeated, rather than counted as a disjoint mathematical sample.

Across development and confirmation, the overlapping edge-case counts include 61 zero-constant models, 323 zero-pole fast queries, 9 trace-one models, 14 trace-minus-one models, 352 queries with $p \mid N$, 416 empty-domain weighted queries and 385 constant-weight queries. The square/nonsquare query counts are 9,318/9,280. All 410 branch single-pole calls and 260 active-branch weighted calls took their explicit unsupported route. Each phase rejected nine invalid controls: composite or out-of-range primes, unsupported degree, repeated roots, zero leading coefficient, a trace outside Hasse's bound, duplicate exclusions modulo p, or a residue-count mismatch. These counters overlap and must not be added to infer independent sample sizes.

The host reported Linux 6.18.55 x86_64 with glibc 2.39 and CPython 3.12.14. The JSON summaries retain the candidate and checker hashes, deterministic seeds and prime lists. The JSONL files retain the input curve, trace, pole or excluded domain, coefficients, direct results and adapter outputs for every mathematical case. The freeze record and later confirmation plan record their order. From the package root, reproduce the checks with

```
python3 verification/verify_revision.py
python3 verification/check_adapter.py --phase development \
  --raw ADAPTER_DEVELOPMENT.jsonl
python3 verification/check_adapter.py --phase confirmation \
  --raw ADAPTER_CONFIRMATION.jsonl
```

These checks concern the new reference adapter and the displayed formulas. Historical campaigns reported above retain their earlier code and data provenance; they were not rerun in this editorial revision. Finite agreement is implementation evidence, not a proof, a novelty audit or a matched performance benchmark.

References

Bostan, A., Gaudry, P., & Schost, É. (2007). Linear recurrences with polynomial coefficients and application to integer factorization and Cartier–Manin operator. *SIAM Journal on Computing, 36*(6), 1777–1806. https://doi.org/10.1137/S0097539704443793

Cosgrave, J. B., & Dilcher, K. (2010). Mod p^3 analogues of theorems of Gauss and Jacobi on binomial coefficients. *Acta Arithmetica, 142*(2), 103–118. https://doi.org/10.4064/aa142-2-1

FLINT developers. (n.d.). ulong_extras.h: Arithmetic and number-theoretic functions for single-word integers, factorials section [FLINT 3.6.0 documentation]. https://flintlib.org/doc/ulong_extras.html

Hart, W. B. (2010). Fast library for number theory: An introduction. In K. Fukuda, J. van der Hoeven, M. Joswig, & N. Takayama (Eds.), *Mathematical software—ICMS 2010* (Lecture Notes in Computer Science, Vol. 6327, pp. 88–91). Springer. https://doi.org/10.1007/978-3-642-15582-6_18

Harvey, D. (2007). Kedlaya's algorithm in larger characteristic. *International Mathematics Research Notices, 2007*, Article rnm095. https://doi.org/10.1093/imrn/rnm095

Harvey, D., & Sutherland, A. V. (2014). Computing Hasse–Witt matrices of hyperelliptic curves in average polynomial time. *LMS Journal of Computation and Mathematics, 17*(A), 257–273. https://doi.org/10.1112/S1461157014000187

Harvey, D., & Sutherland, A. V. (2016). Computing Hasse–Witt matrices of hyperelliptic curves in average polynomial time, II. In *Frobenius distributions: Lang–Trotter and Sato–Tate conjectures* (Contemporary Mathematics, Vol. 663). American Mathematical Society. https://doi.org/10.1090/conm/663/13352

Jordan, D. (2026a, October 18). *Cartier–Miller rounded-third integration: Exact confirmation and workload-matched benchmarks* [Unrefereed computational report and reproducibility package].

Jordan, D. (2026b, October 15). *Rounded third boundary proof and reproducibility companion* [Unrefereed project extension].

Jordan, D. (2026c, October 21). *Cartier–Miller weighted character sums: Primary-source comparisons and matched-output benchmarks* [Unrefereed computational report and reproducibility package].

Jordan, D. (2026d, October 21). *Cartier–Miller uniform curve-order identity and normalization review* [Unrefereed proof and reproducibility package].

PARI Group. (n.d.). *Elliptic curves*. PARI/GP documentation. Retrieved October 4, 2026, from https://pari.math.u-bordeaux.fr/dochtml/ref-stable/Elliptic_curves.html

SageMath Developers. (n.d.). *Frobenius on Monsky–Washnitzer cohomology of a hyperelliptic curve: interval_products* [Sage 10.8 reference manual]. Retrieved October 2, 2026, from https://doc.sagemath.org/html/en/reference/arithmetic_curves/sage/schemes/hyperelliptic_curves/hypellfrob.html#sage.schemes.hyperelliptic_curves.hypellfrob.interval_products

Tal, Y. (2026). *Computing modular factorials below the square-root barrier* [Unrefereed preprint]. ECCC TR26-211. https://eccc.weizmann.ac.il/report/2026/211/

van Wamelen, P. (2002). Jacobi sums over finite fields. *Acta Arithmetica, 102*(1), 1–20. https://doi.org/10.4064/aa102-1-1